DevOps Principles

Build, release and run software as one team: fast flow to customers, fast feedback from them, and a habit of learning.

Engineering Practices · Inspired by The DevOps Handbook (Kim, Humble, Debois, Willis) and Accelerate (Forsgren, Humble, Kim)

TEAM _____

DATE _____

1 Flow

IDEA → CUSTOMER

Speed up the whole path from idea to customer: small batches, visible work, fewer hand-offs, and never passing a known problem downstream.

ASK

"How long does a small change take to reach a customer?"

IN PRACTICE

- Make the work visible, end to end
- Limit work in progress
- Shrink batches and hand-offs

TOOLS IN THIS KIT

- **Value Stream Map**
- **Delivery Pipeline Canvas**
- **Kanban Board Design**

2 Feedback

CUSTOMER → TEAM

Find problems fast and close to where they start: automated tests, monitoring, and teams who see how their work behaves in production.

ASK

"How soon would we know if a release hurt customers?"

IN PRACTICE

- Test automatically, on every change
- Watch what users actually experience
- Fix the cause, then add a check

TOOLS IN THIS KIT

- **Test Strategy**
- **SLOs & Error Budgets**
- **Continuous Delivery Health Check**

3 Continual learning

EXPERIMENT · IMPROVE

Treat failures as information, run small experiments, and set time aside to improve daily work and share what you learn.

ASK

"What did our last incident teach us, and what changed?"

IN PRACTICE

- Blameless reviews after incidents
- Time for improvement, every iteration
- Share lessons across teams

TOOLS IN THIS KIT

- **Blameless Incident Review**
- **1% Experiment Canvas**
- **Retrospective Kit**

CALMS: A Quick Health Check

FIVE PARTS OF DEVOPS · WHICH IS WEAKEST HERE?

PART	WHAT IT MEANS	YOU'LL SEE
C **Culture**	Shared ownership of building and running the work, with trust and no blame.	Developers and operations solve incidents together.
A **Automation**	Automate the repetitive and risky steps: build, test, deploy and infrastructure.	Any change can reach production through one pipeline.
L **Lean**	Small batches, limited work in progress, and less waste on the way to production.	Changes are small, and merged at least daily.
M **Measurement**	Measure flow, quality and outcomes, and use them to learn, not to judge.	The team knows how long changes take and how often they fail.
S **Sharing**	Share tools, knowledge and lessons across teams and roles.	Incident reviews and runbooks are open to everyone.

CALMS was set out by Jez Humble, building on CAMS from John Willis and Damon Edwards. Score each part 1–5 as a team, then improve the lowest.

Four Key Metrics

FROM THE DORA RESEARCH

Deployment frequency	How often changes reach production
Lead time for changes	From commit to running in production
Change failure rate	Share of changes that cause a problem
Time to restore	How fast service recovers after a failure

Speed and stability go together. Track them on Delivery Metrics.

DevOps Habits

TRY ONE THIS ITERATION

You build it, you run it
The team that builds a service helps support it.

Small, frequent changes
Smaller releases are easier to test and undo.

Automate the toil
Turn a repeated manual step into a script.

Blameless by default
Ask what made the mistake easy, not who made it.

"Where does a change wait longest on its way to customers?"

"What would make releasing boring?"

© 2026 Eric Parker · CC BY 4.0 · eparker71.github.io/agile-ways-of-working